

UNIVERSIDAD CATÓLICA DEL URUGUAY



Tarea Obligatoria

Rodrigo Cardinal, Mónica Deus y Ulises Rattin.

Base de Datos I

Cristian Dinardi y Leandro Barral.

23 de noviembre de 2025

Tabla de contenido

Tarea obligatoria.....	3
Fundamentación de diseño.....	3
Herramientas utilizadas.....	4
Visual Studio Code.....	4
Flask.....	4
Werkzeug.....	4
• generated_password_hash toma contraseña de texto sin formato, método hash y longitud de la sal como entrada para producir una contraseña hash. De forma predeterminada, produce una cadena de sal con una longitud de 8.....	5
• check_password_hash toma dos entradas contraseña hash y contraseña de texto sin formato y regresa True si el hash coincide con el hash real de la contraseña de texto sin formato, se devuelve False.....	5
Triggers.....	5
Fundamento de decisiones.....	5
Funcionalidades.....	6
Login.....	6
Salas.....	6
Reservas.....	7
Asistencia.....	8
Sanciones.....	8
Reportes.....	9
Participantes.....	9
Bitácora de trabajo.....	10
Primer encuentro.....	10
Objetivos.....	10
Fecha y responsables.....	10
Actividades realizadas y herramientas utilizadas.....	10
Tareas futuras.....	11
Segundo encuentro.....	11
Objetivos.....	11
Fecha y responsables.....	11
Actividades realizadas y herramientas utilizadas.....	11
Tareas futuras.....	12
Tercer encuentro.....	12
Objetivos.....	12
Fecha y responsables.....	12
Actividades realizadas y herramientas utilizadas.....	12
Tareas futuras.....	13
Cuarto encuentro.....	13
Objetivos.....	13
Fecha y responsables.....	14
Actividades realizadas y herramientas utilizadas.....	14

Tareas futuras.....	14
Quinto encuentro.....	14
Objetivos.....	14
Fecha y responsables.....	15
Actividades realizadas y herramientas utilizadas.....	15
Tareas futuras.....	15
Sexto encuentro.....	15
Objetivos.....	15
Fecha y responsables.....	15
Actividades realizadas y herramientas utilizadas.....	16
Tareas futuras.....	16
Séptimo encuentro.....	16
Objetivos.....	16
Fecha y responsables.....	17
Actividades realizadas y herramientas utilizadas.....	17
Tareas futuras.....	18
Octavo encuentro.....	18
Objetivos.....	18
Fecha y responsables.....	19
Actividades realizadas y herramientas utilizadas.....	19
Tareas futuras.....	19
Noveno encuentro.....	20
Objetivos.....	20
Fecha y responsables.....	20
Actividades realizadas y herramientas utilizadas.....	20
Tareas futuras.....	21
Décimo encuentro.....	21
Objetivos.....	21
Fecha y responsables.....	21
Actividades realizadas y herramientas utilizadas.....	21
Referencias.....	22

Tarea obligatoria

Se nos presentó la propuesta de diseñar e implementar un sistema de información para la gestión de salas de estudio en la universidad. Este sistema tiene como propósito principal optimizar la administración de los espacios disponibles, permitiendo la reserva de salas, el control de asistencia de los usuarios y la generación de reportes que sirvan como herramienta de apoyo para la gestión académica y la toma de decisiones. El trabajo se puede observar en el siguiente repositorio: <https://github.com/RodrigoCardinal/ObligatorioBD1.git>

Fundamentación de diseño

Al momento de crear las tablas, nos basamos en el diseño propuesto en el planteamiento inicial del proyecto, ya que consideramos que representaba de manera adecuada las necesidades del sistema. Sin embargo, realizamos algunas modificaciones, como la creación de la tabla invitado, destinada a usuarios que no pertenecen a la universidad, y la tabla reserva_invitado, que permite que dichos invitados puedan acceder a reservas creadas por los participantes.

Además, en la tabla login incorporamos el atributo es_administrador, cuyo propósito es determinar los permisos disponibles para cada usuario dentro de la aplicación. También agregamos en la tabla reserva el atributo contraseña, con el fin de asegurar que solo las personas que conozcan dicha clave puedan unirse a una reserva ya creada.

Este diseño nos permitió estructurar correctamente la información relacionada con la reserva de salas, el control de asistencia y la gestión de recursos académicos, garantizando una base sólida para el correcto funcionamiento del sistema de gestión de salones.

Herramientas utilizadas

Visual Studio Code

Es un editor de código para programadores gratuito, de código abierto y multiplataforma. Está desarrollado por Microsoft, una compañía con una dilatada experiencia en la creación de IDEs (entornos de desarrollo integrados), que ha conseguido plasmar su larga tradición en el sector para ofrecer una herramienta ligera y práctica que la comunidad ha adoptado en masa (García de Zúñiga, F., 2025).

Flask

De acuerdo a lo establecido por Fernán García de Zúñiga (2025b), Flask es un framework para el desarrollo de proyectos web en Python. Muchos lo calificarían dentro del segmento de los «microframework», ya que tiene un enfoque minimalista y está orientado para un tipo de trabajos más específicos como servicios web y microservicios, en vez del renderizado de HTML como Django.

Werkzeug

Werkzeug es una biblioteca Python que contiene muchas herramientas de desarrollo y depuración para la implementación de la interfaz de puerta de enlace de aplicaciones web. Ésta cuenta con una librería integrada llamada `werkzeug.security`, que cuenta con solamente dos métodos:

- *generated_password_hash* toma contraseña de texto sin formato, método hash y longitud de la sal como entrada para producir una contraseña hash. De forma predeterminada, produce una cadena de sal con una longitud de 8.
- *check_password_hash* toma dos entradas contraseña hash y contraseña de texto sin formato y regresa *True* si el hash coincide con el hash real de la contraseña de texto sin formato, se devuelve *False*.

Triggers

Un trigger o disparador es un script que se usa en lenguaje de programación SQL, en especial en bases de datos como MySQL o PostgreSQL.

Consiste en una serie de reglas predefinidas que se asocian a una tabla. Estas reglas se aplican a la base de datos cuando se realizan determinadas operaciones en la tabla, por ejemplo, al añadir, actualizar o eliminar registros.

Dicho de otra manera, el trigger desencadena determinadas acciones de forma automática en las tablas de la base de datos cuando se insertan, modifican y se añaden nuevos datos.

Fundamento de decisiones

Hemos incorporado un mecanismo de hash seguro de contraseñas utilizando las funciones de la biblioteca Werkzeug, que forma parte del ecosistema de Flask. Gracias a *generate_password_hash* y *check_password_hash*, las contraseñas nunca se almacenan en texto plano: se transforman mediante un algoritmo de derivación de claves con sal¹ incluida y múltiples iteraciones, lo que incrementa notablemente la resistencia frente a ataques de fuerza

¹ Una sal es un fragmento de datos aleatorios que se agrega a una contraseña antes de convertirla en hash y almacenarla.

bruta o compromisos de la base de datos. Esta solución es sencilla de integrar en aplicaciones Flask y ofrece un nivel de seguridad adecuado.

Otro cambio implementado fue impedir la creación manual de sanciones desde la página; ahora solo pueden generarse mediante el trigger *aplicar_sancion*. Esta decisión se tomó por dos motivos principales: automatizar el trabajo del administrador, facilitando su gestión, y evitar que se apliquen sanciones injustificadas.

Funcionalidades

Login

Esta ruta de la aplicación recibe el correo y la contraseña ingresados por el usuario, verifica que el correo exista en la base de datos y comprueba la validez de la contraseña utilizando *check_password_hash*. Además, determina el tipo de participante que está intentando acceder: usuario común, administrador o invitado. Si se trata de un administrador, el sistema le solicita seleccionar el rol con el que desea ingresar; si es un invitado, se valida que el día del intento de acceso coincida con la fecha registrada como día permitido de ingreso. Finalmente, toda la información relevante del usuario autenticado se almacena en la sesión mediante *session*, permitiendo identificar su rol y permisos durante su navegación en el sistema.

Salas

Estas rutas gestionan todo el módulo de salas dentro de la aplicación, permitiendo a los usuarios autenticados consultar el listado de salas, ver su información detallada, revisar horarios disponibles y conocer los turnos ocupados según la fecha seleccionada; además, impiden el acceso a estas páginas a los invitados, redirigiéndolos al inicio. Paralelamente, el sistema incorpora una sección de administración exclusiva para usuarios con rol de

administrador, quienes pueden crear nuevas salas, editar sus datos o eliminarlas, siempre validando campos, revisando la existencia de reservas asociadas y confirmando los cambios mediante mensajes informativos. En todos los casos, se utiliza la información almacenada en la sesión para verificar permisos, y se realizan consultas dinámicas a la base de datos para obtener edificios, tipos de sala, capacidades, imágenes y turnos, asegurando que cada vista muestre datos actualizados y filtrados de acuerdo con los parámetros recibidos. En esta sección, además, el usuario podrá crear las reservas.

Reservas

Este módulo gestiona todo el funcionamiento de las reservas dentro del sistema, permitiendo a los usuarios autenticados listar, filtrar y visualizar reservas, acceder al detalle, crear nuevas o unirse mediante clave, y darse de baja cuando corresponda, mientras que los administradores cuentan con permisos adicionales para editar o eliminar cualquier reserva, ajustar fechas, turnos, salas o estado, y gestionar participantes con validaciones que aseguran la integridad de los datos. En cada operación crítica interviene el verificador, una función central que aplica múltiples controles antes de permitir crear o unirse a una reserva: comprueba que el usuario tenga CI asociado, verifica si posee una sanción activa, valida la compatibilidad entre el tipo de sala y el tipo de usuario — docente, alumno de posgrado o grado —, impide reservas pasadas, controla disponibilidad del turno, y aplica límites semanales y diarios para estudiantes de grado. Al unirse, además, revisa la clave de acceso, la capacidad máxima de la sala y posibles superposiciones de horarios. Todo el flujo depende de la información almacenada en la sesión para determinar permisos, roles y restricciones, garantizando así que tanto usuarios comunes como administradores interactúen con el sistema respetando reglas de disponibilidad, seguridad y uso adecuado de los recursos.

Asistencia

Este módulo permite gestionar y registrar la asistencia de las reservas del día, una función exclusiva para administradores; primero, la ruta principal reúne todas las reservas correspondientes a la fecha actual, obteniendo para cada una su sala, horario y la lista de participantes, de manera que puedan visualizarse fácilmente en el panel de control. Luego, mediante el formulario de marcado, el administrador puede registrar qué participantes asistieron, acción que únicamente está permitida el mismo día de la reserva. El sistema actualiza la asistencia colocando en 1 a los marcados y dejando en 0 al resto, y adicionalmente cambia el estado de la reserva a “sin asistencia” si nadie concurrió. De esta manera, se asegura un seguimiento confiable y actualizado del uso real de las salas, respetando siempre las restricciones de acceso, ya que los invitados y usuarios comunes no pueden acceder a ninguna de estas operaciones.

Sanciones

Este módulo gestiona el sistema de sanciones aplicadas a los participantes, mostrando la información de forma distinta según el tipo de usuario: los administradores pueden visualizar todas las sanciones registradas, mientras que los usuarios comunes solo acceden a las suyas propias. Al listar sanciones, el sistema obtiene desde la base de datos las fechas de inicio y fin de cada sanción, determina si se encuentra activa en la actualidad y la muestra en pantalla junto al nombre del participante. Además, los administradores cuentan con controles avanzados que les permiten editar una sanción existente modificando sus fechas, o eliminarla completamente del sistema; estas operaciones están protegidas con validación de permisos y retroalimentación mediante mensajes flash. En síntesis, este módulo centraliza el control total de las sanciones, asegurando que los usuarios vean únicamente lo que les corresponde y

reservando para los administradores la capacidad de supervisar, corregir y administrar toda la información asociada.

Reportes

Este módulo reúne todos los reportes estadísticos del sistema y está disponible únicamente para administradores, quienes pueden consultar información detallada mediante diferentes filtros y criterios. Al seleccionar un tipo de reporte, el sistema ejecuta consultas adaptadas para obtener datos como las salas más reservadas por edificio, los turnos con mayor demanda según franjas horarias, el promedio de participantes por sala, la cantidad de reservas clasificadas por carrera y facultad, el porcentaje de ocupación de salas por edificio, las reservas y asistencias agrupadas por rol, y las sanciones registradas por tipo de participante. Además, incluye tres reportes adicionales desarrollados por nosotros: el análisis de reservas utilizadas versus no utilizadas, el total de reservas por turno, y las reservas realizadas dentro de un semestre específico. Cada consulta formatea y organiza la información necesaria, incorpora filtros opcionales —como edificio, carrera, rango horario o período de fechas— y envía los resultados listos a la plantilla, permitiendo al administrador evaluar el rendimiento del sistema, identificar patrones de uso y tomar decisiones informadas basadas en datos concretos.

Participantes

Este módulo implementa el ABM (alta, baja y modificación) de participantes e invitados, funciones disponibles exclusivamente para administradores. Desde la vista principal, el sistema obtiene y combina la lista de participantes regulares y de invitados, mostrando además sus responsables y permitiendo seleccionar responsables al crear nuevos registros. Al agregar un participante regular, se validan CI, correo y rol, se guarda su información en las tablas *participante* y *login*, con la contraseña hashada y la opción de

asignarle rol de administrador. En el caso de los invitados, se verifica que el responsable exista, que no haya CI o correo duplicados y se registra la fecha autorizada de ingreso junto con una contraseña temporal. La modificación distingue entre invitados y participantes regulares para evitar cambios indebidos, permitiendo actualizar datos personales, correo, rol o contraseña según corresponda. La eliminación también depende del tipo: se impide borrar participantes con reservas activas o que sean responsables de invitados, y en el caso de los invitados, se revisa que no tengan reservas asociadas. Todas las operaciones incluyen validaciones, manejo de errores y controles de permisos para asegurar que solo administradores puedan gestionar esta información y que los datos se mantengan coherentes.

Bitácora de trabajo

Primer encuentro

Objetivos

- Crear tablas.
- Realizar las inserciones.
- Crear un repositorio GitHub.
- Instalar Flask para la creación de una página web.

Fecha y responsables

Realizado el 08 de octubre de 2025 por Rodrigo Cardinal y Mónica Deus.

Actividades realizadas y herramientas utilizadas

1. Creación de repositorio en GitHub.
2. Discusión sobre la estructura de la base de datos.

3. Creación de la base de datos y de las tablas.
4. Inserciones de datos.²
5. Instalación de Flask.

Tareas futuras

- Definir todas las consultas.
- Ejecutar las consultas.

Segundo encuentro

Objetivos

- Definir y ejecutar todas las consultas

Fecha y responsables

Realizado el 10 de octubre de 2025 por Rodrigo Cardinal y Mónica Deus.

Actividades realizadas y herramientas utilizadas

- Se definieron las consultas agregadas por el equipo:
 - Cantidad de reservas por turnos.
 - Reservas realizadas en el segundo semestre de 2025.
 - Obtener el nombre, apellido y cédula de los participantes que recibieron sanciones, y la cantidad de sanciones asociadas.
- Se realizaron la mayoría de las consultas, con excepción de algunas que la letra resultó confusa.

² Tanto en el paso 3 como en el 4, se utilizó Datagrip.

Tareas futuras

- Terminar las consultas y definir si utilizar Docker o DataGrip.

Tercer encuentro

Objetivos

- Pensar en la estructura para la página web.
- Identificar qué restricciones habrá.
- Decidir si utilizar Docker o Datagrip.
- Realizar la conexión a una base de datos.

Fecha y responsables

Realizado el 26 de octubre de 2025 por Rodrigo Cardinal, Mónica Deus y Ulises Rattin.

Actividades realizadas y herramientas utilizadas

1. Se decidió utilizar DataGrip porque nos resulta más cómodo y ya estamos familiarizados con su uso.
2. Se realizó la conexión a la base de datos.
3. Se identificaron las siguientes restricciones
 - Máximo 2 horas diarias, y no participar más de 3 reservas activas en una semana.
 - No se permite tener más participantes que la capacidad de la sala.
 - Distintos tipos de salas, no se puede reservar una sala que no vaya con tu tipo de participante.

- Crear nueva tabla invitados con inserts de ejemplo.
- Agregar nuevos permisos.

Fecha y responsables

Realizado el 31 de octubre de 2025 por Rodrigo Cardinal, Mónica Deus y Ulises Rattin.

Actividades realizadas y herramientas utilizadas

1. Creamos un login utilizando HTML y Flask.
2. Creamos una nueva tabla para invitados con algunos inserts de ejemplo. Con el objetivo de que participantes externos a la universidad puedan ir a estudiar en las salas, siempre y cuando un miembro de la UCU esté con los invitados en la reserva.
3. Agregamos nuevos permisos, limitando las acciones de los usuarios y otorgando todos los privilegios a los administradores, con el objetivo de mejorar la seguridad de la página.

Tareas futuras

- Continuar trabajando en la página web.
- Evaluar qué otras funcionalidades se le pueden incorporar al proyecto.

Quinto encuentro

Objetivos

- Trabajar en la página web.
- Permitir diferenciar entre usuario común y administrador.
- Agregar trigger sanción.

Fecha y responsables

Realizado el 02 de noviembre de 2025 por Rodrigo Cardinal, Mónica Deus y Ulises Rattin.

Actividades realizadas y herramientas utilizadas

1. Se creó la estructura interna de la web, de manera inicial.
2. Se implementó un cambio en la tabla login en el que se le añadió un atributo donde se analiza si es administrador o no, a través de un booleano.
3. Se creó un trigger que automatiza la creación de sanciones, una vez que se actualiza la tabla reserva, si el estado de esta es 'sin asistencia', se añade en la tabla sanción, las sanciones correspondientes a los participantes.

Tareas futuras

- Continuar trabajando en la página web.
- Evaluar qué otras funcionalidades se le pueden incorporar al proyecto.

Sexto encuentro*Objetivos*

- Trabajar en la página web y en sus funcionalidades.
- Implementar otra herramienta de hasheo de contraseñas.

Fecha y responsables

Realizado el 07 de noviembre de 2025 por Rodrigo Cardinal, Mónica Deus y Ulises Rattin.

Actividades realizadas y herramientas utilizadas

Trabajamos en la página web y en la implementación de sus distintas funcionalidades, centrándonos en mejorar la seguridad y la estabilidad del sistema. Uno de los puntos clave fue incorporar una herramienta de hasheo de contraseñas más robusta, utilizando específicamente las funciones que ofrece *werkzeug.security*, lo que permitió reforzar la protección de los datos de los usuarios. Todo el desarrollo lo realizamos empleando Visual Studio Code, aprovechando sus herramientas y extensiones para agilizar el trabajo y mantener una estructura clara del proyecto. Estas mejoras no solo incrementan la seguridad del sistema, sino que también contribuyen a una experiencia de usuario más confiable y profesional, asegurando que el manejo de credenciales sea adecuado y que el sitio web evolucione hacia un funcionamiento más sólido y sostenible.

Tareas futuras

- Continuar trabajando en la página web.

Séptimo encuentro

Objetivos

- Diferenciamos los permisos de administradores y usuarios.
- Cada vez que se agregue un nuevo usuario lo hashee automáticamente, lo mismo cuando se actualice la contraseña
- Restricciones:
 - Máximo 2 horas diarias, y no participar más de 3 reservas activas en una semana.
 - No se permite tener más participantes que la capacidad de la sala.

- Distintos tipos de salas, no se puede reservar una sala que no vaya con tu tipo de participante.
- Una misma persona no puede tener dos reservas en simultáneo.
- No permitir reservar si se tiene una sanción activa.
- No permitir reservar si la fecha de la reserva ya pasó.
- Permitir unirse y darse de baja a una reserva.
- Ver los turnos de las reservas y su disponibilidad.
- Ver las fotos de las salas en las páginas de detalle.
- Integrarle a las reservas una contraseña para aumentar la seguridad a la hora de unirse.
- Automatizar la actualización del estado de la reserva.
- Arreglar la página ‘Asistencias’ y dejarla funcional.

Fecha y responsables

Realizado el 15 de noviembre de 2025 por Rodrigo Cardinal y Mónica Deus.

Actividades realizadas y herramientas utilizadas

Durante el desarrollo del proyecto realizamos una serie de mejoras orientadas a fortalecer la seguridad, la organización y la eficiencia del sistema de reservas. Se diferenciaron claramente los permisos entre administradores y usuarios, y se incorporó el hashado automático de contraseñas tanto al crear nuevas cuentas como al actualizarlas, garantizando así una mayor protección de los datos. También se aplicaron restricciones de uso, como el límite de dos horas diarias y un máximo de tres reservas activas por semana, además de asegurar que ninguna sala pueda superar su capacidad y que cada participante solo pueda reservar salas compatibles con su tipo. Se evitó que una misma persona tenga reservas simultáneas o pueda reservar con una sanción activa, así como la creación de reservas en fechas ya pasadas. Se añadió

la posibilidad de unirse o darse de baja de una reserva, la visualización de turnos con su disponibilidad y la inclusión de fotos en las páginas de detalle. Para mejorar la seguridad, cada reserva incorpora una contraseña para unirse, y se automatizó la actualización del estado de las reservas. Finalmente, se corrigió y dejó totalmente funcional la página de asistencias. Todas estas funcionalidades se desarrollaron utilizando Visual Studio Code y DataGrip, lo que facilitó tanto la programación como el trabajo con la base de datos. La implementación de estas características aporta beneficios significativos: mejora la experiencia de los usuarios al hacer el sistema más claro y ordenado, reduce errores humanos, aumenta la seguridad de la información y garantiza un funcionamiento más fiable y escalable del sistema en su conjunto.

Tareas futuras

- Implementar funcionalidad para identificar si la persona desea ingresar como administrador o como usuario en el caso de ser administrador.
- Arreglar el trigger sancionador para que cuando se marque la reserva como ‘sin asistencia’, se añada automáticamente la sanción.

Octavo encuentro

Objetivos

- Cambiar la página de inicio.
- Implementar que el usuario únicamente pueda ver sus sanciones.
- ABM de reservas y sanciones.
- Implementar funcionalidad para identificar si la persona desea ingresar como administrador o como usuario en el caso de ser administrador.

- Arreglar el trigger sancionador para que cuando se marque la reserva como ‘sin asistencia’, se añada automáticamente la sanción.
- Conectar los reportes con las consultas.
- Incluir la opción de añadir invitado en la sección de participantes.

Fecha y responsables

Realizado el 16 de noviembre de 2025 por Rodrigo Cardinal y Mónica Deus.

Actividades realizadas y herramientas utilizadas

- Se modificó la página de inicio para mejorar su contenido y estructura.
- Se implementó la funcionalidad que permite que cada usuario visualice únicamente sus propias sanciones.
- Se desarrolló el ABM (Alta, Baja y Modificación) tanto de reservas como de sanciones.
- Se agregó la funcionalidad para permitir que un administrador elija si desea ingresar como usuario común o como administrador al iniciar sesión.
- Se corrigió el trigger sancionador para que, al marcar una reserva como “sin asistencia”, se genere automáticamente la sanción correspondiente.
- Se conectó la sección de reportes con las consultas SQL necesarias para mostrar información en tiempo real.
- Se incorporó la opción de añadir invitados dentro de la gestión de participantes.

Tareas futuras

- Implementar mejoras.
 - Hashear los URL para que no se vean los id de las reservas.

- Que la barra de navegación sólo muestre las secciones a las que el usuario puede acceder.
- Hashear la contraseña de la reserva y la del invitado.
- Realizar un testeo del sistema.
- Terminar el documento y hacer el manual de uso.

Noveno encuentro

Objetivos

- Hashear la contraseña de las reservas una vez que esta se crea.
- Implementar una barra de navegación que muestre sólo lo que el tipo de participante pueda acceder.
- Hashear los URLs de las reservas para que no muestren el ID de las mismas.
- Permitir a los invitados unirse a la reserva.

Fecha y responsables

Realizado el 19 de noviembre de 2025 por Rodrigo Cardinal y Mónica Deus.

Actividades realizadas y herramientas utilizadas

- Se usó `werkzeug.security` para hashear y unhash las contraseñas de las reservas.
- Se permitió que la barra de navegación del administrador muestre todas las secciones, mientras que la del usuario común deja de mostrar *Reportes*, *Asistencia* y *Participantes*, y los invitados únicamente pueden ver las *Reservas*.

- Los URL se hashearón usando Hashids, lo que permitió aumentar la seguridad del sistema.
- Se incorporó una nueva tabla llamada reserva_invitados, destinada a vincular a los invitados con cada reserva. Además, se actualizaron las funcionalidades de la aplicación para permitir que el administrador gestione el ABM de reservas cuando los usuarios sean invitados, y para que dichos invitados puedan unirse o darse de baja de las reservas de forma autónoma.

Tareas futuras

- Terminar la documentación.

Décimo encuentro

Objetivos

- Terminar la documentación.
- Instructivo para correr la aplicación de forma local.

Fecha y responsables

Realizado el 22 de noviembre de 2025 por Rodrigo Cardinal, Mónica Deus y Ulises Rattin.

Actividades realizadas y herramientas utilizadas

- Se creó el instructivo en el ReadMe desde el archivo README.md de VSCode.
- Se terminó la documentación.

Referencias

García de Zúñiga, F. (2025). *¿Qué es Visual Studio Code y cuáles son sus ventajas?* Arsys

<https://www.arsys.es/blog/que-es-visual-studio-code-y-cuales-son-sus-ventajas>

García de Zúñiga, F. (2025b). *¿Qué es Flask (Python) y cuáles son sus ventajas?* Arsys

<https://www.arsys.es/blog/que-es-flask-python-y-cuales-son-sus-ventajas>

Tech Monger,.(2017,26 noviembre).Secure Passwords in Python With Werkzeug

<https://techmonger.github.io/4/secure-passwords-werkzeug/>

Tablado, F. (2020, 22 octubre). *¿Qué es un trigger en una base de datos?* Ayuda Ley

Protección Datos.

<https://ayudaleyprotecciondatos.es/bases-de-datos/trigger/>